

CPT108 Assignment 3:
Weeks 8 to 10

Felix Lianto (2362727)

Instructor: Ho Lam

May 3, 2025

Problem 1

This problem asks us to determine which abstract data structure (ADT) would suit each scenario the best, among these 4 ADTs: linked list, stack, queue, and map.

Scenario 1 ([4]). *You are writing a portion of a program to manage the NFL draft. In the draft teams rotate through 7 rounds of picking in which the team with the worst record for the previous year picks first, followed by the second and so on until the team that won the super bowl picks last. The order of picking is the same in each round. Which ADT would you choose to manage the order of the teams as they rotate through their turn each round?*

Based on this description, we know that there is an ordering to the teams, from worst to best, and that the original order will come around again, as there are 7 rounds of drafting. Therefore, a **queue** would be the best data structure for this, as it can simulate drafting by removing the first (worst) team and pushing it at the end after picking is finished.

Scenario 2 ([4]). *You are writing a program to manage the arrangement of songs at a music festival. Which ADT would you choose to store the order of songs as you build out the show? The process of designing the show will be iterative and you will need the ability to move, add and remove songs before you get to your final design.*

From this, we can tell that the order of the songs are sequential (one song is played before the next), therefore we need to have control over every song in the arrangement. This indicates that the **linked list** data structure is best, as it is traditionally traversed from the head (which, in this case, is the first song) it can handle adding and removing elements (songs) from anywhere within the list.

Scenario 3 ([4]). *You are writing a program to manage unread emails in an inbox. When a new email arrives, it should appear at the very top of the inbox and as the mails are read, they should be removed from the unread view. Which ADT would you choose to manage the ordering of the mails?*

Since new emails are being pushed onto the unread view, it is largely correct to say that the **stack** data structure would prove to be the most useful in this situation. However, there is one caveat, which is that since the stack can only remove the top-most element, the user is strictly confined to reading the newest email sent to them before reading the older ones. This is to say that the stack data structure is not the best fit given that, if their email is riddled with spam, it would be of great annoyance to the user.

Scenario 4 ([4]). *You are writing a program to return search results for an online store. When a user types in a query you should return all products that include the search term in their title. Which ADT would you choose to manage the products and their titles?*

When assigned with browsing a potentially huge amount of entries, similar to this situation about search results from an online store, clearly the data structure implemented must be accommodating to speed when it comes to matching keywords. For that, the **map** data structure will be the most applicable one thanks to its constant look-up time.

Problem 2

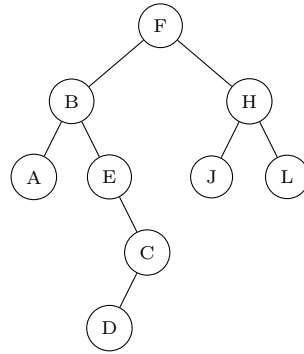


Figure 1: Binary Search Tree for Problem 2

This problem requires us to list out the nodes of the binary tree shown in Figure 1 in pre-, in-, and post-order traversal. Figure 2a, 2b, and 2c shows the order in which the traversal is done. From the figure, we can conclude that the pre-order traversal traverses the letters the following order; **[F, B, A, E, C, D, H, J, L]**. Meanwhile, the in-order traversal orders the value in this order; **[A, B, E, D, C, F, J, H, L]**. Finally, the post-order traversal travels in the order; **[A, D, C, E, B, J, L, H, F]**.

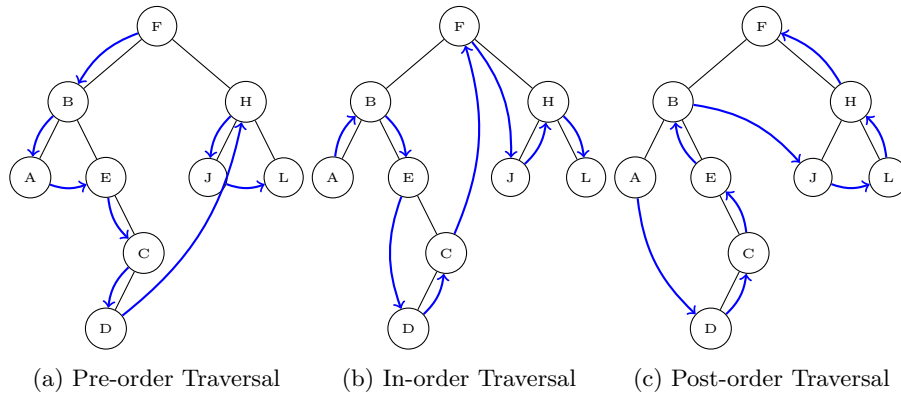


Figure 2: Three types of Binary Search Tree traversals

Problem 3

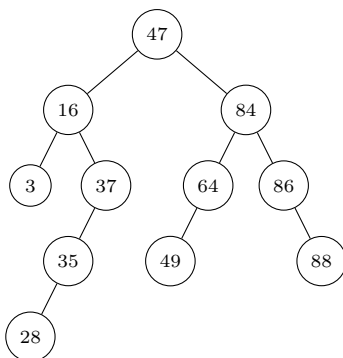


Figure 3: Initial Binary Tree for Problem 3

This problem requires us to perform some operations on this binary tree, and draw its modified tree after each operation. Figure 3 shows the initial tree without any alterations made yet. Figure 4a, 4b, 4c, 4d, and 4e shows the modified tree after these five operations.

Problem 4

For this problem, we are asked to fill in the hash table with a capacity of 11 elements from the 9 elements provided to us in 2 different ways, that is with linear probing and double hashing. To achieve this quickly, C++ code was utilized to visualize what element would go where, in order to minimize error. This part is mainly on the linear probing method, where if a collision appears, the element would be put in the next bucket instead. This is equivalent to a double hashing method, where $h_2(k) = 1$, in which case the hashing function is as simple as $h(k, i) = (h + i) \bmod 11$. The following C++ code in Listing 1 shows the result from the hash function in several values of i , and Figure 5 shows the final resulting hash table from linear probing.

This part will now focus on a double hashing approach to resolving collisions in the hash table. The two hash functions given are $h_1(k) = k$ and $h_2(k) = 1 + (k \bmod 10)$. When put together, the final hash function utilized is of the form $h(k, i) = k + i(1 + (k \bmod 10))$. This function was then implemented as C++ code, which was written to visualize which bucket will each element belong to for 10 different values of i , provided in Listing 2. Figure 6 shows the resulting hash table under the specified hash functions described earlier.

```

1      # include <iostream>
2
3      int main() {
4          int values[] { 10,22,31,4,15,28,17,88,59 };
5          for (int v : values) {
6              std::cout << v << '\t';
7          }
8          std::cout << "\n\n";
9          for (int i = 0; i < 10; i++) {
10             for (int k : values) {
11                 std::cout << (h + i) % 11 << "\t";
12             }
13             std::cout << '\n';
14         }
15
16         return 0;
17     }

```

Listing 1: C++ code to print out the linear probed indices for various i .

Problem 5

To determine which hash function is the best for distributing 2020 keys into 10 buckets, we will need to examine each function with a statistical test to determine the closest one to uniformity. To achieve this, we will utilize the Chi-Squared Goodness-of-fit Test, given as the formula;

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (1)$$

where O_i denotes the observed value and E_i denotes the expected value [2]. Since we are looking for uniformity, the most ideal case would be that each bucket receives the same amount of keys, which we can find by dividing 2020 by 10, which would give us $E_i = 202$ for every bucket i .

Now, to obtain our observed values, we can do a bit of programming to calculate these values fast. This can be achieved by setting an array of 10 elements, initialized at 0, and then for each of the 2020 keys, we can compute the hashed value $h(k)$ and increment the value at the index $h(k) \bmod 10$. The counter array will give us the amount of elements placed into each bucket, thereby giving us the observed values. Listing 3 gives the C implementation of this algorithm.

Using the code provided, we can accurately plot the observed values on a line graph shown in Figure 7. Graphically, we can see from the figure that the **hash function C** provides the most even distribution out of the 4. This is in line with our Chi-Squared statistical test as well, as hash function C outputs the smallest χ^2 at 109.430693, followed by function B and D with 1616, and

```

1      # include <iostream>
2
3      int h1(int k) {
4          return k;
5      }
6
7      int h2(int k) {
8          return 1 + (k % 10);
9      }
10
11     int main() {
12         int values[] { 10,22,31,4,15,28,17,88,59 };
13         for (int v : values) {
14             std::cout << v << '\t';
15         }
16         std::cout << "\n\n";
17         for (int i = 0; i < 10; i++) {
18             for (int k : values) {
19                 std::cout << (h1(k) + i * h2(k)) % 11 << "\t";
20             }
21             std::cout << '\n';
22         }
23
24         return 0;
25     }

```

Listing 2: C++ code to print out the double hashed indices for various i .

finally, the most uneven, function A at 2020.

Problem 6

For this problem, we are given pseudocode for removing duplicates within an array, as shown in Algorithm 1. From the pseudocode given, it is clear that the complexity will rely on the performance of searching for an item in the set and inserting that item into the set. If the set implemented was a binary search tree, then the time complexity of both searching and inserting is **linear time** $O(h)$, where h is the height of the tree [1]. Indeed, in the worst case where the tree is heavily lopsided, the total complexity would equal to that of a linked list, which is $O(n)$, and in the average case scenario, it would be some multiple of 2, meaning $O(\log n)$ [1]. The total runtime of this algorithm is, accounting for the fact that appending to a dynamic array costs constant time [5], would be $O(2n \log n) \in \boxed{O(n \log n)}$ on average, taking the for-loop into consideration.

Meanwhile, if the set implemented was a hash table, searching and inserting would only take a time complexity of **constant time** $O(1)$ on average [3].

```

1      int main() {
2          int n = 10;
3          int counter[n];
4
5          int k;
6          for (k = 0; k < n; i++) {
7              counter[k] = 0;
8          }
9
10         int maxK = 2020;
11         for (k = 0; k < maxK; i++) {
12             counter[h(k) % n]++;
13         }
14     }

```

Listing 3: Counting algorithm to produce the observed values.

Of course, collisions might still appear and, in the worst case, slow insertion down to $O(n)$, but regardless, this function would run marginally faster if the set implemented was a hash table [3], as this provides the total average time complexity of $O(n)$ for the algorithm.

Algorithm 1: Pseudocode provided in Problem 6.

Function *removeDuplicate(A)*:

```

    // A: Input Array
    S = new empty set;
    R = new empty dynamic array;
    foreach  $x \in A$  do
        if not  $S.contains(x)$  then
            S.add(x);
            R.append(x);
        end
    end
end

```

Problem 7

This problem is an extension of the famous LeetCode Two Sum problem, where we need to find a pair of numbers within the array that adds up to a target number. This problem has a special, further restriction of 'closeness', where we can only search for elements that are sufficiently close enough to each other. More specifically, their absolute distance to each other cannot be less

```

1      #include <vector>
2      #include <unordered_map>
3
4      bool hasClosePair(std::vector<int> B, int r) {
5          int n{ static_cast<int>(B.size()) };
6          int wLen{ n / 10 };
7          std::unordered_map<int, int> lookup{};
8
9          for (int i{ 0 }; i < n; i++) {
10             int d{ r - B[i] };
11
12             if (lookup.contains(d)) {
13                 int j{ lookup.at(d) };
14                 if (abs(i - j) < wLen) return true;
15             }
16
17             lookup[B[i]] = i;
18         }
19
20         return false;
21     }

```

Listing 4: C++ implementation of the algorithm for part A

than a tenth of the total elements in the array. We can achieve this by utilizing the hash table to provide the average case $O(n)$ time complexity algorithm the problem asks for.

To start, we will need to initialize a hash table to store a mapping between the elements in the array and its index. Then, for every element in the array, we will take the difference d between it and the target number r , and we check to see if that difference is in the table. If it is, then we have found the pair of numbers that can sum to the target, since

$$\begin{aligned}
 r - B_i &= B_j \\
 B_i + B_j &= r
 \end{aligned}$$

where $j \neq i$.

Then, we can compare their indices by taking the absolute difference and return true if it is lower than $n/10$, rounded down. If either the element isn't in the hash table or the indices are too far apart, then we add the key-value pair i and B_i , where B_i is the key, into the hash table for the next iteration to compare to. If nothing matches, then the loop will exit and return false. The C++ implementation of this algorithm can be found in Listing 4. Clearly, inserting into the table will take an expected, amortized $O(1)$ complexity, while look-ups will also take an expected $O(1)$ complexity. Since we are running through the loop n times, it is expected that the final runtime complexity of

this algorithm falls into the expected $O(n)$ time complexity.

Now, to achieve a worst-case $O(n)$ algorithm would normally be impossible, but with the addition of a new restriction on r , that is $r < n^2$, we can achieve this by utilizing direct addressing. Normally, direct addressing would be memory unsafe for a very large r , given that the hash array to keep track of your values could be unreasonably large. But since r is now bounded by n^2 , we can utilize this method. We start by initializing an array of size $r + 1$, initialized with the value -1 . This ensures that when checking for closeness, we are comparing it against values already in the table rather than uninitialized values. Then, just like before, we will take the difference between r and B_i , but now we will compare it between 0 and r inclusive, to ensure no out-of-bounds index error. Afterwards, we check if the value is initialized or not with $H_i > -1$ where H is the hash table and i is the current index, and if it is close enough with $\text{abs}(i - H_i) < wLen$. If all 4 conditions are met, then the search concludes and returns true. Otherwise, it runs through one more check to see if the index B_i is less than or equal to r , to which the hash table updates to set its index i at B_i if true. The C++ implementation can be seen in Listing 5. Since every value

```

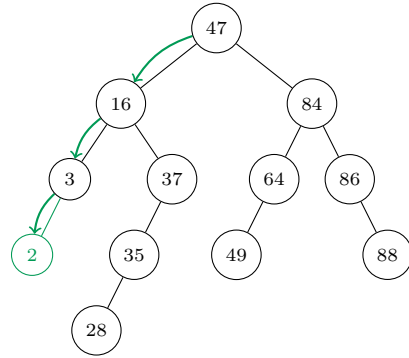
1      #include <iostream>
2      #include <vector>
3
4      bool hasClosePair(std::vector<int> B, int r) {
5          int n{ static_cast<int>(B.size()) };
6          int wLen{ n / 10 };
7          std::vector<int> lookup(r+1, -1);
8
9          for (int i{ 0 }; i < n; i++) {
10             int d{ r - B[i] };
11
12             if (d > -1 && d < r+1) {
13                 if (lookup[d] > -1 && abs(i - lookup[d]) < wLen) return
14                     ↪ true;
15             }
16
17             if (B[i] <= r) {
18                 lookup[B[i]] = i;
19             }
20
21             return false;
22     }

```

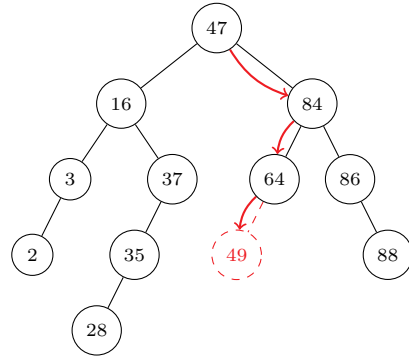
Listing 5: C++ implementation of the algorithm for part B.

in the array is unique, and the modulo was not utilized thanks to the direct addressing scheme for the hash table, we are not required to do additional work

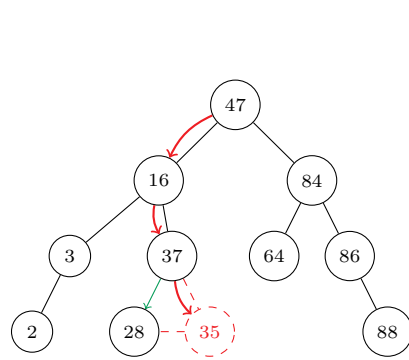
for collisions because it is impossible for one to appear. Thus, this algorithm has a runtime of $O(1)$ in the worst case, due to the time complexity of a direct addressing hash table look-up and insertion.



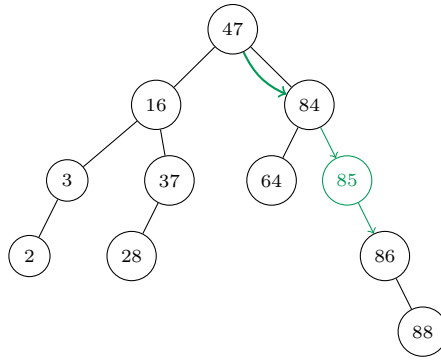
(a) Binary Tree after **inserting 2**



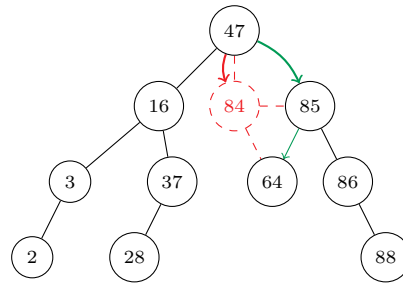
(b) Binary Tree after **deleting 49**



(c) Binary Tree after **deleting 35**



(d) Binary Tree after **inserting 85**



(e) Binary Tree after **deleting 84**

Figure 4: Binary Trees after each modifications

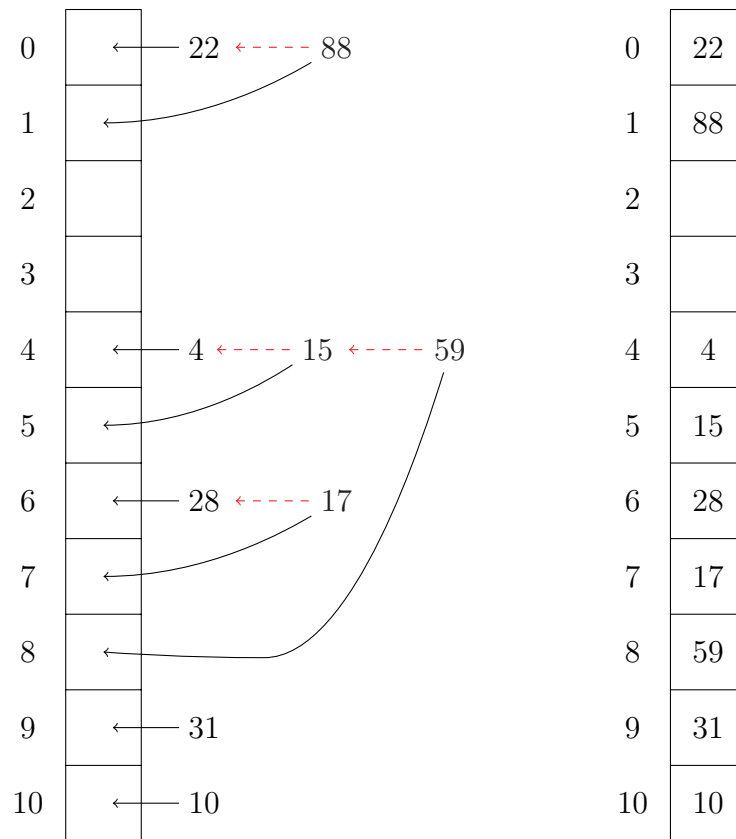


Figure 5: Linear Probing; how each element is distributed in the hash table (left) and the final occupied hash table (right)

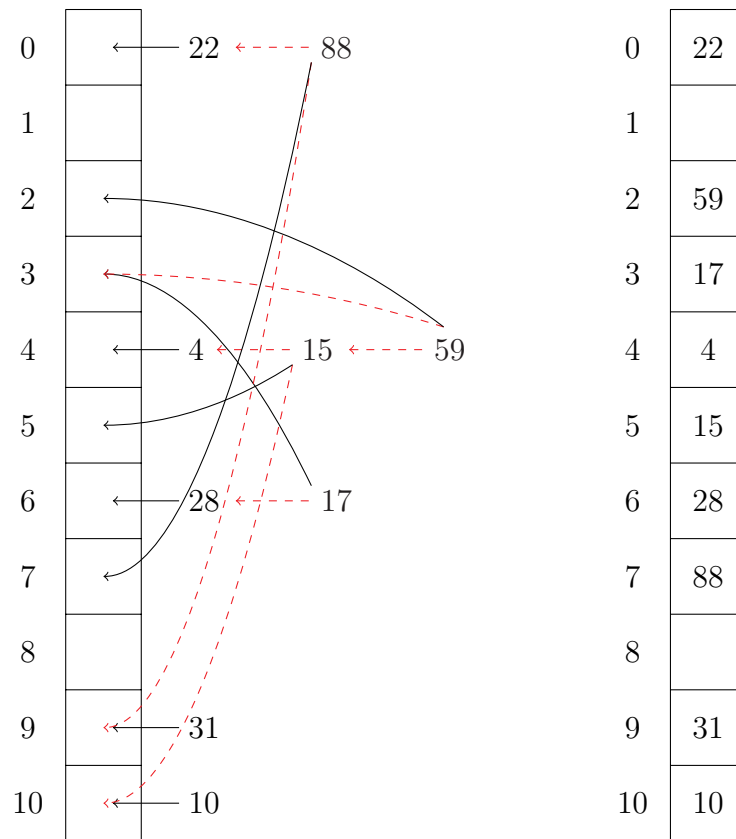


Figure 6: Double Hashing; how each element is distributed in the hash table (left) and the final occupied hash table (right)

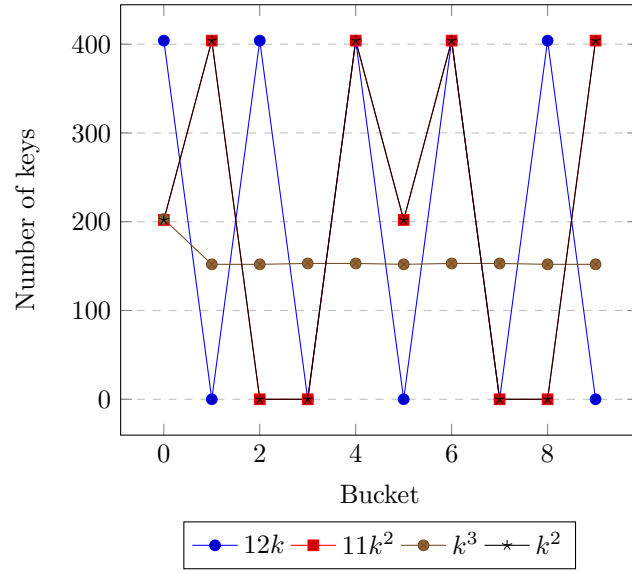


Figure 7: Distribution of keys over the buckets

Bibliography

- [1] *Binary Search Tree*. In: *Wikipedia*. May 2, 2025. URL: https://en.wikipedia.org/w/index.php?title=Binary_search_tree&oldid=1288395034 (visited on 05/02/2025).
- [2] *Goodness of Fit*. In: *Wikipedia*. Sept. 20, 2024. URL: https://en.wikipedia.org/w/index.php?title=Goodness_of_fit&oldid=1246721899 (visited on 05/02/2025).
- [3] *Hash Table*. In: *Wikipedia*. Mar. 28, 2025. URL: https://en.wikipedia.org/w/index.php?title=Hash_table&oldid=1282794260 (visited on 05/02/2025).
- [4] Ho Lam. *Assignment 3: Weeks 8 to 10*. Suzhou, China: Xi'an Jiaotong-Liverpool University. 7 pp. URL: https://core.xjtlu.edu.cn/pluginfile.php/168318/mod_assign/introattachment/0/assignment03_week8_10.pdf?forcedownload=1 (visited on 05/02/2025).
- [5] ptin. *Answer to "Why Is the Time Complexity of Appending Items at the End of the Dynamic Array $O(1)$ "*. Stack Overflow. Dec. 20, 2023. URL: <https://stackoverflow.com/a/77688797> (visited on 05/02/2025).